

CPS 2231 Computer Programming

Chapter 5: Polymorphism in Java

Dr. Hamza Djigal

Assistant Professor,

Department of Computer Science,
Wenzhou-Kean University

dhamza@kean.edu

Semester: Fall 2025

November 2, 2025



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

- 1 **Objective**
- 2 Polymorphism in Java
- 3 Compile-Time Polymorphism (Method Overloading)
- 4 Method Overloading and Type Promotion
- 5 Runtime Polymorphism (Method Overriding)
- 6 Rules for Method Overriding
- 7 Upcasting and Downcasting
- 8 Practice

Objectives

At the end of this chapter, you will be able to:

- Define the types of polymorphism.
- Create and use method overloading.
- Create and use method overriding.
- Differentiate compile-time and run-time polymorphism.

- 1 Objective
- 2 Polymorphism in Java**
- 3 Compile-Time Polymorphism (Method Overloading)
- 4 Method Overloading and Type Promotion
- 5 Runtime Polymorphism (Method Overriding)
- 6 Rules for Method Overriding
- 7 Upcasting and Downcasting
- 8 Practice

Polymorphism in Java

Definition

Polymorphism in Java is a concept by which a single action can be performed in different ways.

Example: A person at the same time can have different characteristics: *he can be a father, a husband, and an employee.*

```
1 class Person {
2     void role() {System.out.print("General role");}
3 }
4 class Father extends Person{
5     void role() {System.out.print("I'm a father");}
6 }
7 class Husband extends Person {
8     void role() {System.out.print("I'm a husband");}
9 }
10 class Employee extends Person {
11     void role() {System.out.print("I'm an employee");}
12 }
```

Polymorphism in Java

```
1 public class Main {
2     public static void main(String[] args) {
3         Person p;
4
5         p = new Father();
6         p.role(); // I am a father
7
8         p = new Husband();
9         p.role(); // I am a husband
10
11        p = new Employee();
12        p.role(); // I am an employee
13    }
14 }
```

Type of Polymorphism in Java

Two types of polymorphism

- Polymorphism is achieved via **method overloading** and **method overriding**.
- Two types of polymorphism in Java:
 - Compile-time polymorphism: Also called **static polymorphism**. It is achieved by **method overloading**.
 - Runtime polymorphism: or Dynamic Method Dispatch is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

- 1 Objective
- 2 Polymorphism in Java
- 3 Compile-Time Polymorphism (Method Overloading)**
- 4 Method Overloading and Type Promotion
- 5 Runtime Polymorphism (Method Overriding)
- 6 Rules for Method Overriding
- 7 Upcasting and Downcasting
- 8 Practice

Compile-Time Polymorphism

Method Overloading

Allows a class to have more than one method with the same name, but with **different parameters**.

Different ways to overload a method

- By changing the number of arguments:

```
1 class Calculator {  
2     int add(int a, int b){return a + b;}  
3     int add(int a, int b, int c){return a + b + c;  
4     }  
5 }  
6 public class Test {  
7     public static void main(String[] args) {  
8         Calculator calc = new Calculator();  
9         System.out.println(calc.add(2, 3));  
10        System.out.println(calc.add(1, 2, 3));    }  
11    }
```

Compile-Time Polymorphism

Different ways to overload a method

- By changing the data type of arguments:

```
1 class Calculator {
2     int multiply(int a, int b) {return a * b;}
3
4     double multiply(double a, double b) {return a *
5         b;}
6 }
7 public class Test {
8     public static void main(String[] args) {
9         Calculator calc = new Calculator();
10        System.out.println(calc.multiply(3, 4));
11        System.out.println(calc.multiply(2.5, 4.0))
12            ;
13    }
14 }
```

Compile-Time Polymorphism

What is not possible in Method Overloading

- Method overloading is **not possible by changing only the return type.**

```
1 class Calculator {
2     int add(int a, int b) { return a + b; }
3     // This will cause a compile-time error!
4     double add(int a, int b) { return a + b; }
5 }
6
7 public class Test {
8     public static void main(String[] args) {
9         Calculator calc = new Calculator();
10        System.out.println(calc.add(2, 3)); //
11                                           Error: method add(int, int) is already
12                                           defined
13    }
14 }
```

Compile-Time Polymorphism

What is not possible in Method Overloading

- Overloading by changing only the access modifiers:

```
1 class Calculator {
2     public int add(int a, int b) { return a + b; }
3     // This will cause a compile-time error!
4     private int add(int a, int b) { return a + b; }
5 }
6
7 public class Test {
8     public static void main(String[] args) {
9         Calculator calc = new Calculator();
10        System.out.println(calc.add(2, 3)); //
11                                           Error: method add(int, int) is already
12                                           defined
13    }
14 }
```

Compile-Time Polymorphism

What is not possible in Method Overloading

- Overloading by changing only the access modifiers:

```
1 class Calculator {
2     public int add(int a, int b) { return a + b; }
3     // This will cause a compile-time error!
4     private int add(int a, int b) { return a + b; }
5 }
6
7 public class Test {
8     public static void main(String[] args) {
9         Calculator calc = new Calculator();
10        System.out.println(calc.add(2, 3)); //
11        Error: method add(int, int) is already
12        defined
13    }
14 }
```

Compile-Time Polymorphism

Summary of Rules for Method Overloading

Possible Ways to Overload a Method:

- By changing the number of parameters.
- By changing the data type of parameters.
- By changing the order of parameters.

Not Possible Ways to Overload a Method:

- By changing only the return type.
- By changing only the access modifiers.

- 1 Objective
- 2 Polymorphism in Java
- 3 Compile-Time Polymorphism (Method Overloading)
- 4 Method Overloading and Type Promotion**
- 5 Runtime Polymorphism (Method Overriding)
- 6 Rules for Method Overriding
- 7 Upcasting and Downcasting
- 8 Practice

Type Promotion

- **Type Promotion** arises when a smaller data type is automatically converted to a larger compatible type to match a method parameter. This often comes into play in method overloading.
- Java performs **type promotion** automatically if it finds that the passed argument type is compatible with the method parameter type.

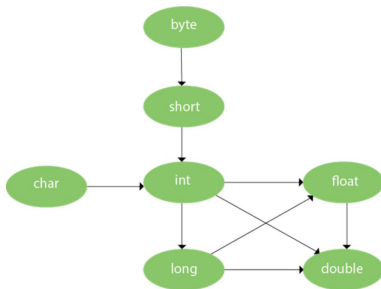


Figure 1: Type promotion in Java

Type Promotion

```
1  class Calculator {
2      void multiply(int a, int b) {
3          System.out.println("int multiply: "+(a * b));}
4
5      void multiply(double a, double b) {
6          System.out.println("double multiply: "+(a * b)
7              );}
8  }
9  public class Test {
10     public static void main(String[] args) {
11         Calculator calc = new Calculator();
12         calc.multiply(5, 3); //match: int multiply
13         calc.multiply(5.0, 3.0); //match: double
14         calc.multiply(5, 3.0); //Type promotion: int to
15         calc.multiply(5.0f, 3); //Type promotion: float
16         calc.multiply(5, 3.0); //Type promotion: int to double
17     } }
```

Overloading Example with Argument Types

Overloading Example with Argument Types

```
1  class Printer {  
2      void print(int a) { System.out.println(a); }  
3      void print(double a) { System.out.println(a); }  
4      void print(String s) { System.out.println(s); }  
5  }  
6  public class TestPrinter {  
7      public static void main(String[] args) {  
8          Printer p = new Printer();  
9          p.print(100);  
10         p.print(3.14);  
11         p.print("Hello");  
12     }  
13 }
```

Varargs in Method Overloading

Definition

In Java, **Varargs (variable-length arguments)** is a mechanism that allow a method to accept zero or more arguments of a specified type. They are declared using an ellipsis (...).

Example

```
1 void print(int... nums) { } //This method can  
    accept any number of int arguments e.g., print  
    (1), print(1,2,3)
```

Varargs in Method Overloading

Normal Overloading with Varargs

```
1  class Test {  
2      void show(int a) {  
3          System.out.println("Single int argument");  
4      }  
5  
6      void show(int... a) {  
7          System.out.println("Varargs method");  
8      }  
9  
10     public static void main(String[] args) {  
11         test ex = new Test();  
12         ex.show(10); // Calls the first method  
13                     with single int argument  
14         ex.show(10, 20); // Calls the second method  
15     }
```

Varargs in Method Overloading

Varargs with Different Data Types

```
1 class Display {  
2     void show(String msg, int... nums) {  
3         System.out.println("String and int varargs"  
4             );  
5     }  
6     void show(String... msgs) {  
7         System.out.println("String varargs only");  
8     }  
9  
10    public static void main(String[] args) {  
11        Display d = new Display();  
12        d.show("Hello", 1, 2, 3);  
13        d.show("Hi", "There");  
14    }  
15 }
```

Varargs in Method Overloading

Important Rules

- Varargs must be the last parameter in the method signature.

```
1 void print(int... nums, String msg); //Invalid
2 void print(String msg, int... nums); // Valid
```

- Ambiguity can occur if multiple overloaded methods can match the same call.

```
1 void test(int... a) { }
2 void test(Integer... a) { }
3
4 test(1, 2, 3); // Compile-time error: ambiguous
                  method call
```

- If an exact match exists, Java prefers it over a varargs version.
- You can't overload solely by changing the varargs type if the compiler can't distinguish between them.

- 1 Objective
- 2 Polymorphism in Java
- 3 Compile-Time Polymorphism (Method Overloading)
- 4 Method Overloading and Type Promotion
- 5 Runtime Polymorphism (Method Overriding)**
- 6 Rules for Method Overriding
- 7 Upcasting and Downcasting
- 8 Practice

Method Overriding

Definition

If a subclass has a method with the **same name and parameters** as in the parent class, it is **overriding** the parent method.

- Used to provide a **specific implementation** of a method already defined in the superclass.

Rules for Method Overriding:

- Method name must be the same.
- Parameter list must be the same.
- Return type must be compatible (covariant return allowed).
- Access level cannot be more restrictive.
- Cannot override `final` or `static` methods.

Method Overriding Example

```
1 // Parent class
2 class Animal {
3     void sound() { System.out.println("Animal makes
4         a sound"); }
5 }
6 // Child class
7 class Dog extends Animal {
8     void sound() { System.out.println("Dog barks");
9     }
10 }
11 public class TestAnimal {
12     public static void main(String[] args) {
13         Animal a = new Dog(); // runtime
14         a.sound();             // Dog barks
15     }
16 }
```

- 1 Objective
- 2 Polymorphism in Java
- 3 Compile-Time Polymorphism (Method Overloading)
- 4 Method Overloading and Type Promotion
- 5 Runtime Polymorphism (Method Overriding)
- 6 Rules for Method Overriding**
- 7 Upcasting and Downcasting
- 8 Practice

Rules for Method Overriding

1. Method Name Must Be the Same

```
1 class Parent {  
2     void display() { System.out.println("Parent  
        display"); }  
3 }  
4 class Child extends Parent {  
5     @Override  
6     void display() { System.out.println("Child  
        display"); } // Same name  
7 }
```

Explanation: The method name must match exactly.

2. Parameter List Must Be the Same

```
1 class Parent { void show(int a) { } }  
2 class Child extends Parent {  
3     @Override  
4     void show(int a) { } // Same parameters
```

Rules for Method Overriding

3. Covariant Return Type (Compatible Return Type)

```
1 class Animal { Animal getAnimal() { return new
    Animal(); } }
2 class Dog extends Animal {
3     @Override
4     Dog getAnimal() { return new Dog(); } //
        Covariant return
5 }
```

Explanation: The overridden method can return a subclass type.

4. Access Level Cannot Be More Restrictive

```
1 class Parent { protected void greet() { } }
2 class Child extends Parent {
3     @Override
4     private void greet() { } //More restrictive
5 } // Cannot reduce visibility (protected to private
    )
```

Rules for Method Overriding

5. Cannot Override `final` Methods

```
1 class Vehicle {
2     public final void start() {
3         System.out.println("Start");
4     }
5 }
6 class Car extends Vehicle {
7     @Override
8     public void start() { } // Error
9 }
```

Explanation: `final` methods cannot be overridden.

Rules for Method Overriding

6. Static Methods Are Hidden, Not Overridden

```
1 class Parent {  
2     static void info() {  
3         System.out.println("Parent");  
4     }  
5 }  
6 class Child extends Parent {  
7     static void info() {  
8         System.out.println("Child"); } // Hides method  
9  
10 }
```

Explanation: static methods are class-level; they are hidden, not overridden.

Rules for Method Overriding

7. Override Annotation Is Optional But Recommended

```
1 class Parent {  
2     void display() { System.out.println("Parent  
        display"); }  
3 }  
4 class Child extends Parent {  
5     @Override // This annotation is optional, but  
        it helps catch errors  
6     void display() { System.out.println("Child  
        display"); }  
7 }
```

Explanation: The Override annotation is not strictly required, but it ensures the method matches the one in the superclass and helps catch errors.

Rules for Method Overriding

8. Method Overriding Works with Dynamic (Runtime) Polymorphism

```
1  class Parent {
2      void display() { System.out.println("Parent
           display"); }
3  }
4  class Child extends Parent {
5      @Override
6      void display() { System.out.println("Child
           display"); }
7  }
8  public class Main {
9      public static void main(String[] args) {
10         Parent p = new Child();
11         p.display(); // Outputs: Child display
12     }
13 }
```

Explanation: The method executed depends on the object's class.

Rules for Method Overriding

11. Overriding Works in Interfaces

```
1 interface Animal {  
2     void sound();  
3 }  
4 class Dog implements Animal {  
5     @Override  
6     public void sound() { System.out.println("Bark"  
7         ); }  
}
```

Explanation: If a class implements an interface, it must override the methods defined in that interface (unless the class is abstract).

Rules for Method Overriding

11. Overriding in Abstract Classes

```
1 abstract class Animal {  
2     abstract void sound();  
3 }  
4 class Dog extends Animal {  
5     @Override  
6     void sound() { System.out.println("Bark"); }  
7 }
```

Explanation: An abstract class can have abstract methods. A concrete subclass must override these abstract methods.

Compile-Time vs Run-Time Polymorphism

- **Compile-time polymorphism:** Method overloading, resolved at compile time.
- **Run-time polymorphism:** Method overriding, resolved at runtime via dynamic method dispatch.

Definition

Upcasting: When a reference variable of a *parent class* refers to an object of its *child class*, it is known as **upcasting**. It is used to achieve **runtime polymorphism** (method overriding) in Java.

- 1 Objective
- 2 Polymorphism in Java
- 3 Compile-Time Polymorphism (Method Overloading)
- 4 Method Overloading and Type Promotion
- 5 Runtime Polymorphism (Method Overriding)
- 6 Rules for Method Overriding
- 7 Upcasting and Downcasting**
- 8 Practice

Upcasting in Java

Example: Upcasting

```
1 class Parent {
2     void show() {System.out.println("Parent class
3         method");}
4 }
5 class Child extends Parent {
6     @Override
7     void show() {System.out.println("Child class
8         method");}
9     void display() {System.out.println("Child
10        specific method");}
11 }
12 public class UpcastingExample {
13     public static void main(String[] args) {
14         Parent obj = new Child();//Upcasting
15         obj.show();//Calls Child's overridden
16             method (runtime polymorphism)
17         // obj.display(); // Not allowed: Parent reference
18             can't access child methods.
```

Upcasting in Java

Explanation

- `Parent obj = new Child();` → Upcasting: the parent reference points to a child object.
- The `show()` method is overridden in `Child`, so the child version executes at runtime.
- **Upcasting** allows dynamic method dispatch but limits access to parent class methods only.
- The **reference type of obj is Parent**, even though it points to a `Child` object.
- The **compiler only checks** methods defined in the reference type (`Parent`).
- Since `display()` is not defined in the `Parent` class, the compiler cannot find it.

Methods vs Data Members in Runtime Polymorphism

Note

- A method is **overridden**, not the data members, so **runtime polymorphism** cannot be achieved by data members.
- When a parent class reference refers to a child object, the **method** call depends on the **object type** (runtime), but the **data member** access depends on the **reference type** (compile time).

Methods vs Data Members in Runtime Polymorphism

Java

```
1  class Parent {int value = 10;
2      void show() {System.out.println("Parent show()
3          method"); }
4  }
5  class Child extends Parent {
6      int value = 20;
7      @Override
8      void show() {System.out.println("Child show()
9          method");}
10 }
11 public class PolymorphismExample {
12     public static void main(String[] args) {
13         Parent obj = new Child(); // Upcasting
14         System.out.println("Value: " + obj.value);
15         // output 10
16     }
17 }
```


Java instanceof Operator

Definition

The `instanceof` operator tests whether an object is an **instance of a specific class**, subclass, or interface.

Syntax

```
object instanceof ClassName
```

Example

```
1 class Animal {}
2 class Dog extends Animal {}
3 public class InstanceOfDemo {
4     public static void main(String[] args) {
5         Animal a = new Dog();
6         System.out.println(a instanceof Animal); //true
7         System.out.println(a instanceof Dog); //true
8         System.out.println(a instanceof Object); // true
9     }
10 }
```

Java instanceof Operator

Key Points

- Returns `true` if object is an instance of the given type.
- Returns `false` if the reference is `null`.
- Helps ensure safe type casting in polymorphism.

Downcasting with instanceof

Definition

Downcasting is the process of converting a *superclass* reference back into a *subclass* type.

It allows access to the subclass' s specific methods or fields, but must be done carefully.

The instanceof operator is used to check whether the object is actually an instance of that subclass before performing the cast; to avoid a `ClassCastException`.

```
1 class Animal {  
2     void makeSound() {System.out.println("Some  
        generic animal sound");}  
3 }  
4  
5 class Dog extends Animal {  
6     void bark() {System.out.println("Woof!");}  
7 }
```

Downcasting with instanceof

Example: Safe Downcasting using instanceof

```
1 public class DowncastingDemo {
2     public static void main(String[] args) {
3         Animal a = new Dog(); // Upcasting: Dog ->
4                                 Animal
5         // Safe Downcasting
6         if (a instanceof Dog) {
7             Dog d = (Dog) a; // Downcasting
8             d.bark(); // Access subclass method
9         }
10        else {System.out.println("Not a Dog instance!");}
11    }
12 }
```

Downcasting with `instanceof`

Output

Woof!

Key Points

- Upcasting is always safe: from subclass to superclass.
- Downcasting requires type checking with `instanceof`.
- Prevents runtime errors and ensures safe polymorphic behavior.

- 1 Objective
- 2 Polymorphism in Java
- 3 Compile-Time Polymorphism (Method Overloading)
- 4 Method Overloading and Type Promotion
- 5 Runtime Polymorphism (Method Overriding)
- 6 Rules for Method Overriding
- 7 Upcasting and Downcasting
- 8 Practice**

Time to Practice

Scenario

We want a program that works with different shapes, such as **triangle**, **rectangle**, etc.

- Design a superclass called `Shape`, which defines a method called `getArea()` that returns the area of the shape.
- Create subclasses `Rectangle` and `Triangle` that inherit from `Shape` and override the `getArea()` method.
- Write a main class to test your code.

Step-by-Step Tasks

- 1 **Shape.java:** Create the superclass with a `getArea()` method that returns 0 by default.
- 2 **Rectangle.java:** Create a subclass with fields `width` and `height`, constructor, and override `getArea()`.
- 3 **Triangle.java:** Create a subclass with fields `base` and `height`, constructor, and override `getArea()`.

Time to Practice

Step-by-Step Tasks

- ➊ **ShapeTest.java:** Create objects of `Rectangle` and `Triangle`, store them in a `Shape[]` array, loop through the array, and print the areas.
- ➋ **Optional:** Use `instanceof` to print the type of each shape.

Hint

- Use method overriding for each subclass to compute the correct area.
- Demonstrate polymorphism by storing different shapes in a single array of type `Shape`.